Table of Contents

Smaky 6 — The Desperate Programmer's Manual **1**
Chapter 0: Philosophy 1
Chapter 1: Starting Up — or "Why It Won't Boot" 1
Chapter 2: The Command Line — or "Git Doesn't Exist Yet, and It's Wonderful" 2
Chapter 3: File Management — or "Your Entire Filesystem Fits in 315 KB" 2
Chapter 4: File Extensions — or "Under 3 Letters and Proud of It" 3
Chapter 5: Function Keys — or "Seven Keys to Rule Them All" 3
Chapter 6: Peripherals — or "$LP Is Not a Rapper" 4
Chapter 7: Error Codes — or "ERROR 043 and Me" 5
Chapter 8: The Monitor — or "Welcome to Comfortable Hell" 5
Chapter 9: Sound — or "Beep" 5
Chapter 10: FAQ — Frequently Asked Questions 6
Epilogue 7

Smaky 6 — The Desperate Programmer's Manual

For those who thought computers would be easy.

> This manual assumes you have already survived at least one DOS crash, one Linux kernel panic, or one software requirements meeting.

Chapter 0: Philosophy

The Smaky 6 was designed in a blessed era when:

- "Copy-paste" literally meant copying and pasting (with actual paste)
- 64 KB of RAM was **considered an obscene luxury**
- The concept of "crash" did not exist — programs *gracefully terminated* as a tasteful `ERROR 043`
- There were no graphics drivers to update, no Windows Update rebooting at the worst moment, and absolutely **no one** had opinions about this month's JavaScript framework

If your program doesn't fit in 64 KB, that's your problem. If it does fit in 64 KB, that's also your problem, but an *elegant* one.

Chapter 1: Starting Up — or "Why It Won't Boot"

Step 1: Insert the floppy disk into DX0:

This is the hardest part for a modern developer. A *floppy disk* is a circular physical device that stores data magnetically. It doesn't plug in via USB. It has no Bluetooth driver. It doesn't require a cloud account. It does require **inserting in the correct orientation** — a skill lost since 1998.

Step 2: Press SHIFT-BREAK

Note that the key is called `BREAK`, not `Ctrl+Alt+Del`, not `sudo reboot`, not `systemctl reboot --force --force`. Just `SHIFT-BREAK`. Cherish this simplicity.

Step 3: Wait for the > prompt

If you see `ERROR 043`, return to Step 1. If you see `ERROR 033`, return to Step 1 with a different disk. If you see nothing, check that the screen is powered on. (Don't laugh. Support tickets have been filed for this reason.)

```
ROM de chargement rev 1-7      ← The system says hello
SAMOS  rev 2-8                 ← The OS says hello
DX0:                           ← The drive says hello
>                              ← Your keyboard says "your turn"
```

Chapter 2: The Command Line — or "Git Doesn't Exist Yet, and It's Wonderful"

The **>** prompt is the ancestor of your favourite terminal. It has no oh-my-zsh plugin. It doesn't show the git branch in the prompt. It has no Dracula theme. It's just **>**.

It's beautiful.

The killer shortcut: the `TAB` key

On your modern computer, `TAB` completes paths. On the Smaky 6, `TAB` directly inserts `DX1:` into the command line — because the most common use case was copying files to the second drive, and the designers decided not to waste your time.

Compare this with `npm` autocomplete which takes 3 seconds to load.

The `BREAK` key cancels the current line. If the line is already empty, it recalls the last command. This is exactly `Ctrl+C` and `↑` combined into a single key. Maximum efficiency. Minimum heart rate.

Chapter 3: File Management — or "Your Entire Filesystem Fits in 315 KB"

`LIST` — List files

```
> LIST
```

Shows all your files. There are few of them. That's intentional. No `node_modules`. No `.git` with 47,000 pack objects. No mysterious `__pycache__` folder that keeps coming back.

On the Smaky 6, if you have 20 files, you're a power user.

Modern equivalent: `ls -la` followed by 47 columns of which you read 2.

`COPY SOURCE DEST` — Copy a file

```
> COPY MYFILE.SM DX1:
```

Copies a file. It copies the file. That's all it does. It doesn't create 18 cached versions. It doesn't ask if you want to sync to the cloud. It doesn't suggest installing an extension.

It copies the file.

Take a moment to appreciate this.

DOS equivalent: `COPY` *(same thing, Smaky had it first) Unix equivalent:* `cp` *(same thing with fewer letters)*

`DELETE FILE` — Delete a file

```
> DELETE SHAME.BS
```

Deletes the file. **Permanently.** No recycle bin. No confirmation. No "Are you sure? (y/n)" — which is simultaneously terrifying and refreshing.

Note: If the file is marked **permanent** (`ERROR 4`), the system politely refuses to delete it. That's the only protection that exists. Cherish it.

Modern equivalent: `rm -rf` but for wise people who don't have wildcard accidents.

`COMPRESS` — Compact the disk

```
> COMPRESS
```

Reorganises the disk to reclaim space from deleted files. Think of it as the 1980s garbage collector.

Philosophical note: In 2026, garbage collectors do this automatically in the background on your 32 GB of RAM while you watch YouTube. On the Smaky 6, you had to do it manually, which gave you a **sense of total control** and a vague existential satisfaction.

Chapter 4: File Extensions — or "Under 3 Letters and Proud of It"

The Smaky 6 uses 2-letter extensions, because its designers knew that life is short and floppy disks are small.

Extension	Meaning	Programmer's commentary
`.SY`	System file	Do not delete. No, seriously. Do not delete.
`.SM`	Executable (program)	The `.exe` of the era, but *without* UAC dialogs
`.MC`	Macro (CLI script)	Shell scripts, without 40 years of `bash` quirks
`.BS`	BASIC source	The language that taught computing to a generation
`.SR`	SMILE assembler source	For those who found BASIC too "high level"
`.DR`	Directory (folder)	A folder that is also a file. Very zen.
`.LS`	Assembler listing	The ancestor of `make 2>&1 | tee build.log`
`.ST`	Symbol table	What `nm` and `objdump` do, but less chatty
`.IM`	1-bpp bitmap image	Resolution is low, but so are expectations
`.HP`	Help file	Help that actually helps — a revolutionary concept

---

Chapter 5: Function Keys — or "Seven Keys to Rule Them All"

The Smaky 6 has **7 physical function keys** engraved on the keyboard. Each program assigns them whatever meaning it wants.

Compare with your modern keyboard which has 12 function keys of which you only use `F5` (refresh), `F11` (fullscreen), and occasionally `F2` (rename, when you remember).

Key	PC equiv	What it does in theory	What it does in practice
`CHANGE`	F1	Change something	Depends on the program
`SEARCH`	F2	Search for something	Depends on the program
`SHOW`	F3	Show something	Depends on the program
`COPY`	F4	Copy something	Depends on the program
`CURSOR`	F5	Move the cursor	Depends on the program
`PROGRA`	F6	Program something	Depends on the program (very useful)
`KILL`	F7	**Kill something**	Aborts a transfer. Perfect name.

The `KILL` key is the only one with consistent behaviour everywhere. It kills things. That's its job. It's honest.

Chapter 6: Peripherals — or "$LP Is Not a Rapper"

Peripherals are referenced with a `$` followed by 2 letters. It's the environment variable of the era, but useful.

```
> COPY REPORT.BS $LP          — Print to printer
> COPY $PR PROGRAM.SR         — Read a paper tape
> COPY DATA.BS $MO            — Send data via 300-baud modem
```

`$LP` — Line Printer

Prints on a real noisy dot-matrix printer that goes "TRRRTRRRTRRR" and whose every page smells of ink and hope. Requires `LP.SY` loaded.

Modern equivalent: Sending to the office printer and hoping nobody plugged it into a Mac on a different network.

`$PR` / **`$PP`** — Paper tape reader/punch

Paper tape is the ancestor of USB. Except it doesn't plug in — it unrolls. And if you drop it you spend an hour winding it back up. Connection via 20 mA current loop, USART 4.

Status in 2026: Absolutely nobody uses this. Absolutely nobody.

`$MI` / **`$MO`** — Modem

300 to 2400 baud. Enough to transfer your program in 20 minutes. Enough to hear "squeeee KRRR PING PONG squeeee" and call it "connecting".

Note: The modem was optional. Patience was mandatory.

Chapter 7: Error Codes — or "ERROR 043 and Me"

Errors are displayed in **octal**. Because why not.

If you grew up with modern error messages like:

```
TypeError: Cannot read properties of undefined (reading 'map')
    at Object.<anonymous> (/app/node_modules/some-lib/dist/index.js:1:42837)
    ... 47 lines of stack trace in minified code ...
```

…then `ERROR 043` will seem crystal clear to you.

Quick translation table:

What you see	What it means	What you should do
`ERROR 043`	Bad load	Check the disk. Retry. Pray.
`ERROR 033`	Read error	The disk is probably damaged
`ERROR 032`	Disk full	Delete things you don't love
`ERROR 031`	Write protect tab set	Remove the write-protect tab
`ERROR 014`	File does not exist	Check spelling. Usually that's it.
`ERROR 013`	File already exists	Pick a different name
`ERROR 004`	Permanent file	This file does not want to die
`ERROR 001`	Write protect file	The file says "no"

Important note on octal: `ERROR 043` = octal 43 = decimal 35 = *Bad load*. Not decimal 43. Not hex 43. **Octal.** Welcome to the 1980s, where the number base was a style choice.

Chapter 8: The Monitor — or "Welcome to Comfortable Hell"

```
> MON
```

This command launches **SYSMON**, the machine monitor. It's the Z80 equivalent of `gdb`, except you work in hexadecimal, in octal, and sometimes in the hope of having become an accountant instead.

The SYSMON lets you: - Read and modify any byte of RAM (all RAM, no permissions, no sudo) - Execute machine code directly - Debug your programs by inspecting registers one at a time - Feel like you actually control the machine

For modern developers: it's like having direct access to `/proc/mem` but **without segfaults**, without AppArmor, and without the kernel OOM-killer terminating your session at the worst moment.

To exit the monitor: type the monitor's exit command. *(Yes, we're not documenting it here. Discovering the exit command is part of the initiation.)*

Chapter 9: Sound — or "Beep"

The Smaky 6 has a **programmable speaker**.

It goes "beep".

More precisely, the ROM code performs about 96 writes to port 0x03 at ~2139 cycle intervals, producing a square wave at ~584 Hz for ~82 ms.

In modern terms: it's a notification sound that cannot be disabled, cannot be muted, and cannot be replaced by a vibration. It's either "beep" or silence.

Modern users who complain about notification sounds have never experienced 1982 mechanical keyboards.

To produce sound from your program:

```
?BEEP       — a standard beep
?PLAY       — a melody (frequency/duration table, terminated by 0)
```

Modern equivalent: `console.log('\x07')` (ASCII BEL — doesn't work in most modern terminals anymore, proof that progress is not always linear)

Chapter 10: FAQ — Frequently Asked Questions

Q: My program doesn't fit in 64 KB. What should I do?

A: Rewrite the program. Remove features. Learn to count bytes. Reconsider your life choices.

Q: How do I access the internet from the Smaky 6?

A: You can't. This is a feature.

Q: I accidentally deleted `SYS.SY`. How do I recover?

A: You don't. This is why we make backups. *Welcome to 1980s computing.*

Q: The program crashes with `ERROR 043`. What debugger should I use?

A: `MON`. Then your eyes. Then the Z80 instruction set reference. Then possibly graph paper to draw the call stack by hand.

Q: How do I install additional packages?

A: You copy files from another floppy disk. That's package management. No `npm install` downloading 847 MB of transitive dependencies.

Q: The floppy is making a strange noise. Is that normal?

A: No. Back up now. Back up everything. Back up twice.

Q: Can I run Docker on the Smaky 6?

A: Docker won’t exist for another 30 years. Enjoy the peace.

Epilogue

The Smaky 6 was designed by people who loved computing and wanted computing to be **useful**, **understandable**, and **fast** — at a time when “fast” meant “responds in under a second on hardware you can repair yourself with a soldering iron”.

Forty years later, our machines are a million times more powerful and our web pages take 8 seconds to load.

The Smaky 6 has no opinion on this. It awaits your command.

>

Manual written with affection for a machine that deserves better than a museum.